Music API

A React music application with an Express API, external catalogue and audio providers, a shared playback engine, and MongoDB-backed library records. The implementation is designed for local demonstration rather than public music distribution.

Application responsibilities

Frontend. React, Vite and React Router render Home, Search, New, Radio, Podcasts, album and artist pages, plus library views. AppLayout maintains the audio element and current track, queue, timing, volume, shuffle and repeat state.

Backend. Express routes call service-layer functions. Provider results are mapped to application objects. An in-process Map cache supports selected read endpoints with a default ten-minute expiry.

Application data. Mongoose models represent user, playlist and track records. The current user service accesses a shared user record; this is not a multi-user authenticated service.

Provider integrations

Deezer: primary track, artist, album and category searches; provider-supplied preview audio URLs. These do not imply rights to host or stream complete commercial recordings.

iTunes and RSS: podcast search and metadata from iTunes; episode lists and media enclosure URLs from publisher feeds.

Radio Browser: a station lookup adapter exists. The current Radio view also uses artist-led Deezer selections assembled by the content service.

Spotify: a client-credentials search adapter exists in source but is not invoked by the primary music service. It should not be represented as the live catalogue integration.

Data path

React page or floating player → Express route → provider adapter / Mongoose → mapped JSON response → shared playback and queue state

Source: github.com/keenosmith-del/music-api | Based on repository README, routes, controllers, services and React implementation.

Playback and persistence

The main engineering decisions concern where to keep audio state, how to handle third-party API data, and which user actions are durable.

Playback state

AppLayout contains the audio ref and shared state for current track, playback position, duration, volume, mute, queue, shuffle, repeat and autoplay. The floating player and expanded player receive that state rather than maintaining separate playback sessions. Changing routes preserves player continuity within the layout.

Queue logic works with mapped track objects. Its next, previous and autoplay behaviour depends on source availability and valid preview URLs. Live media availability and buffering were not independently benchmarked.

Persistence and cached data

The MongoDB routes support favourite tracks, a library and playlist records. The provider data cache is a process-local JavaScript Map with an expiry timestamp; it does not persist across restarts or coordinate multiple backend instances.

The frontend also retains some catalogue responses in React context and browser storage. These caches are separate from the Mongoose library records.

Review of implementation boundaries

Authentication: the current user API routes do not apply authentication middleware, and the client uses a local signed-in flag. This is not sufficient for distinct user accounts or public access.

Provider reliability: upstream APIs, preview links, RSS media and radio streams can change, rate-limit or become unavailable. Public operation needs explicit error/fallback tests and provider terms review.

Secrets and deployment: the repository tree includes a tracked backend/.env. Review the file outside public logs and remove/rotate any secrets before deployment. No production deployment status is established by source inspection.

Verification: application files and implementation paths were inspected. No committed automated test suite or CI workflow was found in this revision; build, browser playback and integration results are not claimed as independently passed.

Technology used

React, Vite, React Router, Context, Framer Motion, Node.js, Express, Axios, Deezer, iTunes Search, RSS, Radio Browser, MongoDB and Mongoose. Spotify exists as an unconnected adapter.